INPUT:-

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *left, *right;
};

// Create a new node
struct node* create(int value)
{
   struct node *newnode;
   newnode = (struct node*)malloc(sizeof(struct node));

   newnode->data = value;
   newnode->left = NULL;
   newnode->right = NULL;

   return newnode;
}

// Insert element in BST
struct node* insert(struct node *root, int value)
{
   if (root == NULL)
       return create(value);

   if (value < root->data)
       root->left = insert(root->left, value);
   else if (value > root->data)
```

```
        root->right = insert(root->right, value);

    return root;
}

// Search an element
void search(struct node *root, int key)
{
    if (root == NULL)
    {
        printf("Element not found\n");
        return;
    }

    if (root->data == key)
        printf("Element found\n");
    else if (key < root->data)
        search(root->left, key);
    else
        search(root->right, key);
}

// Inorder Traversal
void inorder(struct node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```c
// Preorder Traversal
void preorder(struct node *root)
{
   if (root != NULL)
   {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

// Postorder Traversal
void postorder(struct node *root)
{
   if (root != NULL)
   {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}

int main()
{
   struct node *root = NULL;
   int n, value, key, i;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
```

```
    {
        printf("Enter value: ");
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("\nInorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);

    printf("\nEnter element to search: ");
    scanf("%d", &key);

    search(root, key);

    return 0;
}
```

OUTPUT:-

```
Enter number of nodes: 5
Enter value: 50
Enter value: 30
Enter value: 70
Enter value: 20
Enter value: 40

Inorder: 20 30 40 50 70
Preorder: 50 30 20 40 70
Postorder: 20 40 30 70 50
Enter element to search: 40
Element found
```